Time Series Project: IPI Food Industries

1 Part I: The data

1.1 1. What does the chosen series represent?

The dataset is an Industrial Production Index (IPI) for the food industries sector in France, with monthly frequency, seasonally and working-day adjusted (CVS-CJO), base 100 in 2021. The variable is an index and is unit-free. Since the series is already CVS-CJO, we do not apply additional seasonal adjustment.

Modelling context.

- As an index (base 100 in 2021), values above (below) 100 indicate production above (below) its 2021 average level for the sector.
- The CVS-CJO correction removes most deterministic seasonal patterns and calendar effects; consequently, we do not expect pronounced remaining seasonality and we focus on non-seasonal ARIMA/ARMA dynamics.
- Because the index is strictly positive over the sample, the analysis can be carried out on $Z_t = \log(Y_t)$; first differences of Z_t have a growth-rate interpretation and often provide variance stabilization.

1.2 2. Transform the series to make it stationary if necessary

We read the monthly values file, which contains multiple vintages. Each vintage is stored as a pair of columns: a value column followed by a code column (A, P, R, etc.). The row labeled "Mises a jour" gives the update date for each vintage. We select the most recent vintage by default.

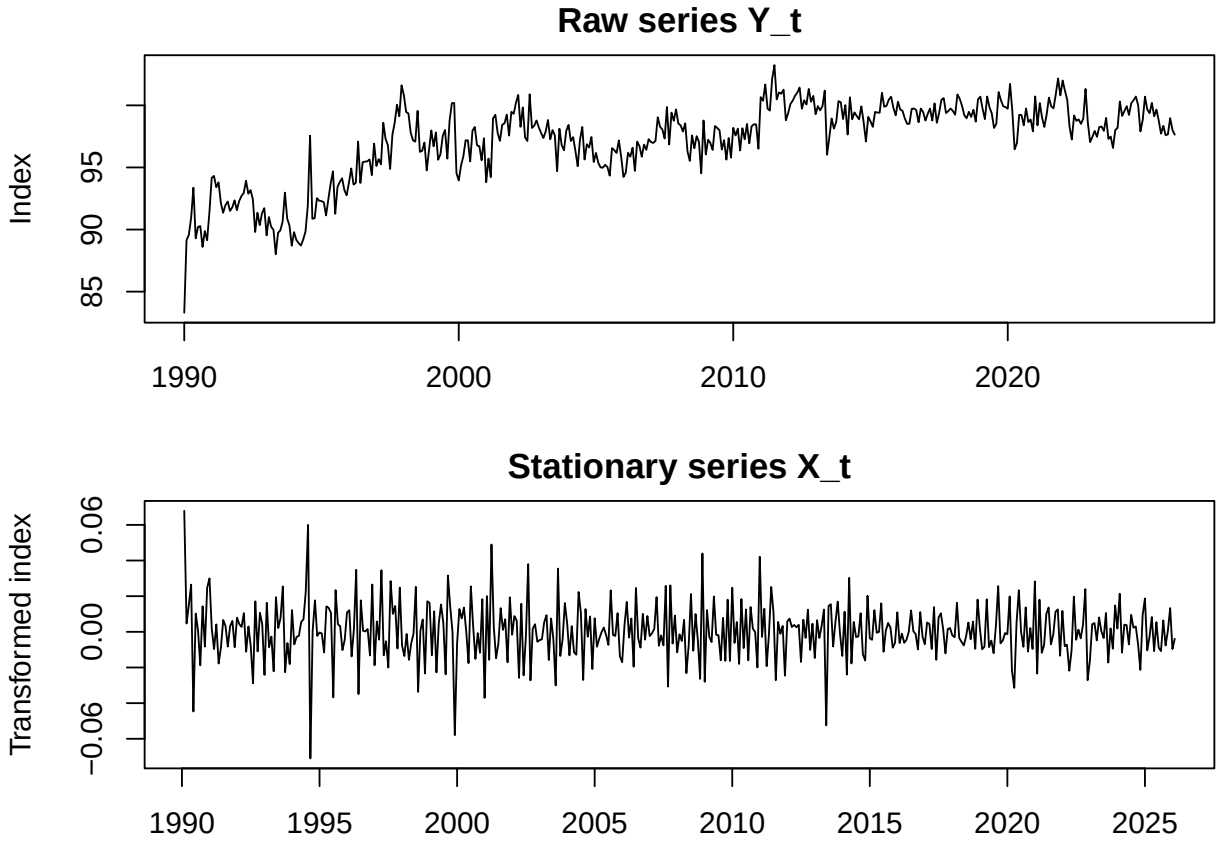
We consider a log transform if all values are strictly positive. Then we determine the number of differences needed using unit-root tests and verify stationarity with ADF and KPSS.

Justification.

- **Log transform.** Since the series is strictly positive over the sample, we work with $Z_t = \log(Y_t)$. Then $\Delta Z_t = \log(Y_t) - \log(Y_{t-1}) \approx (Y_t - Y_{t-1})/Y_{t-1}$.
- **Sample size and frequency.** The series is monthly with 434 observations, from 1990-1 to 2026-2. This satisfies the "at least 100 observations" requirement and is long enough for ARMA identification.
- **Stationarity of Z_t.** The ADF test on Z_t has p-value 0.309, so it does not reject a unit root at the 5% level. The KPSS test (level-stationarity null) on Z_t has p-value 0.01, so it rejects stationarity. Taken together, the tests support the presence of a stochastic trend (non-stationarity) in Z_t.
- **Choosing the differencing order d.** `forecast::ndiffs()` suggests 0 difference(s) under ADF and 1 under KPSS, hence we take the conservative choice $d = \max(d_{ADF}, d_{KPSS}) = 1$.
- **Stationarity of $X_t = \nabla^d Z_t$.** After differencing, ADF on X_t yields p-value 0.01 (reject unit root) while KPSS yields p-value 0.1 (do not reject stationarity). This is consistent with X_t being stationary.

We therefore define the stationary modelling series as $X_t = \nabla^1 \log(Y_t)$.

1.3 3. Graphical representation before and after transformation



Graphical assessment.

- On the raw index Y_t, the level is persistent (slowly moving), which is typical of a non-stationary macro index; large macro events can create visible breaks in level and volatility.
- On $X_t = \Delta \log(Y_t)$, the series fluctuates around a stable mean and shocks are short-lived, which is consistent with a stationary specification.

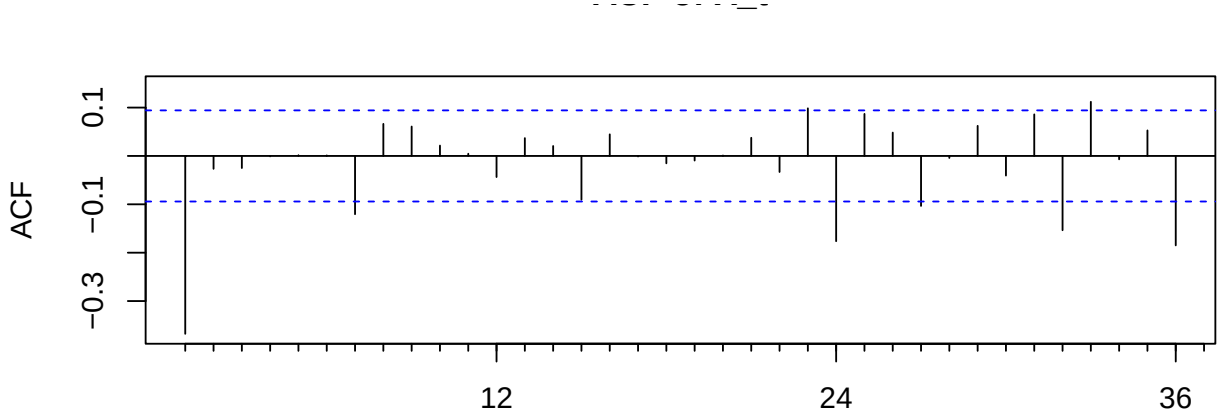
2 Part II: ARMA models

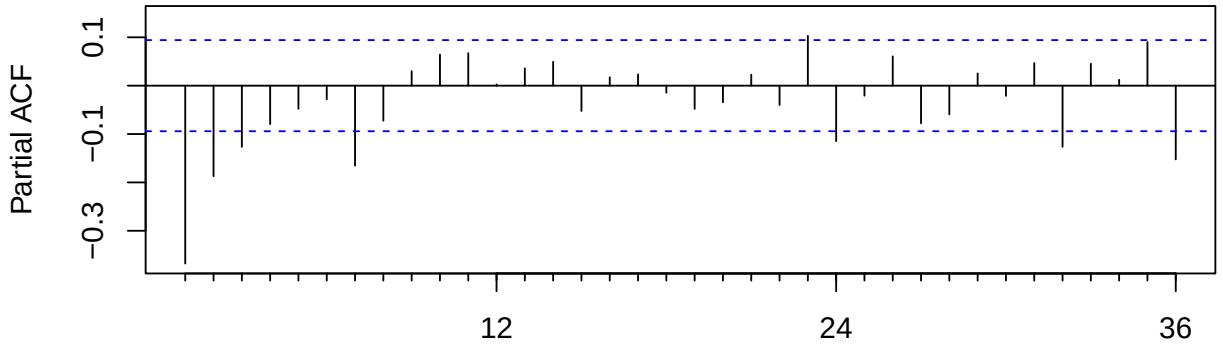
2.1 4. Pick and estimate an ARMA(p,q) model for X_t

We use ACF/PACF diagnostics and an information-criterion search on a small grid to select a parsimonious ARMA model.

Model identification and selection.

- **Identification logic.** The ACF/PACF of X_t are used as qualitative guidance: short-range dependence suggests an ARMA with relatively small orders rather than a high-order AR (pure PACF cutoff) or pure MA (pure ACF cutoff).
- **Quantitative selection.** We then compare ARMA(p, q) candidates on a grid $(0 \leq p, q \leq 5)$ and select the feasible model minimizing AIC. AIC is preferred here because the goal is forecasting/fit (it penalizes complexity less harshly than BIC).





p	q	aic	bic
4	5	-2477.383	-2432.605
5	5	-2475.193	-2426.344
3	3	-2472.856	-2440.290
4	3	-2471.407	-2434.770
3	4	-2471.403	-2434.766

p	q	aic	bic
4	5	-2477.383	-2432.605

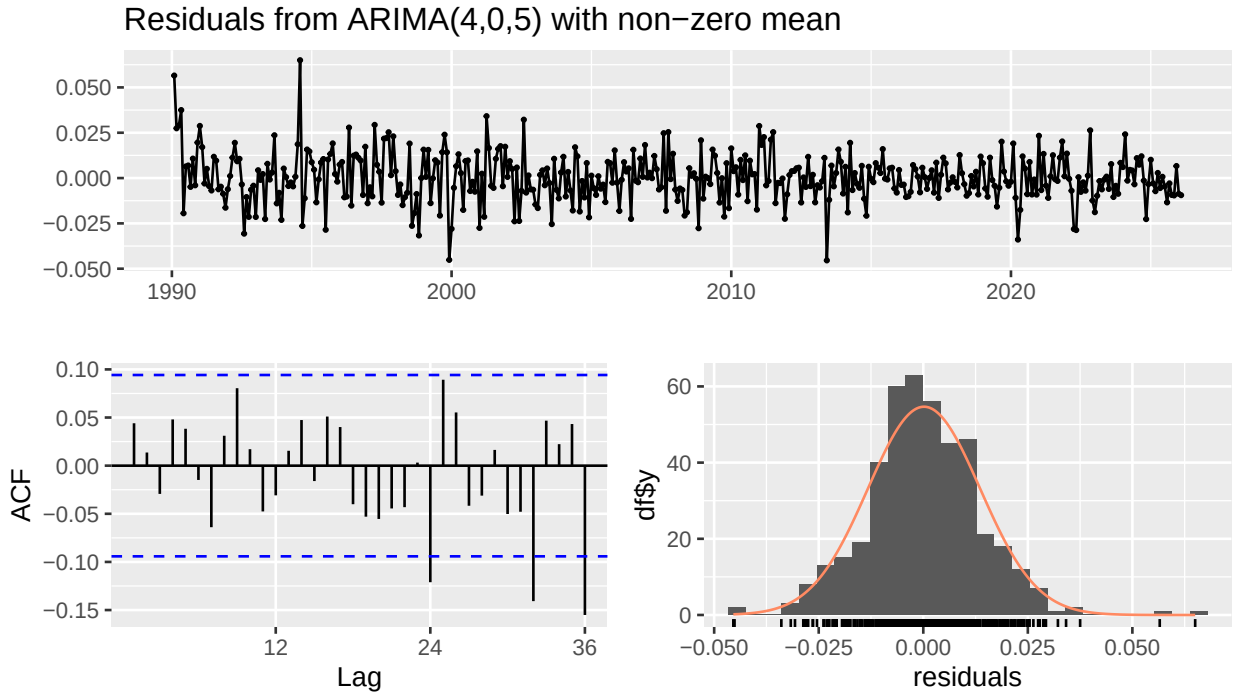
We select the model with the lowest AIC among feasible candidates, which yields ARMA($p = 4, q = 5$) for X_t. The estimated parameters are:

term	estimate	std_error	t_value
ar1	0.0541	0.0161	3.3653
ar2	0.6039	0.0196	30.8225
ar3	0.0132	0.0267	0.4954
ar4	-0.9645	0.0157	-61.5563
ma1	-0.6250	0.0570	-10.9711
ma2	-0.6444	0.0457	-14.0972
ma3	0.3517	0.0640	5.4990

term	estimate	std_error	t_value
ma4	0.9961	0.0483	20.6066
ma5	-0.6083	0.0708	-8.5912
intercept	0.0003	0.0002	1.0646

2.1.1 Diagnostic checks

We validate the fitted model via residual autocorrelation, Ljung-Box test, and a Gaussianity check.



Residual diagnostics.

- **Whiteness.** The Ljung–Box test checks for remaining autocorrelation in residuals. A non-significant p-value supports the ARMA adequacy (no leftover linear dependence).
- **Gaussianity.** The Jarque–Bera test is a strict check for normality. For many macro series, residuals can be non-Gaussian because of outliers (e.g., crisis months), which affects distributional statements (prediction intervals/regions) more than point forecasts.
- **Here (numbers).** Ljung–Box at lag 24 (df adjusted) gives p-value 0.0529, and Jarque–Bera gives p-value 1.89×10^{-15}.

Overall, the model is considered acceptable for linear dependence if residual autocorrelation is not significant; the Gaussian assumption used later for the confidence region should be interpreted as an approximation if normality is rejected.

2.2 5. Write the ARIMA(p,d,q) model for the chosen series

Let Z_t be the transformed series (log if used) and $X_t = \nabla^d Z_t$ be the stationary series. If the selected ARMA model is ARMA(p, q) for X_t, then the original series follows an ARIMA(p, d, q) model for Z_t. If $Z_t = \log Y_t$, this corresponds to an ARIMA(p, d, q) for the log series, which implies a multiplicative structure on Y_t.

ARIMA representation. With $d = 1$ and the selected ARMA($p = 4, q = 5$) for $X_t = \nabla^d Z_t$, the model can be written as

$$\phi(B)\,\nabla^d Z_t = c + \theta(B)\,\varepsilon_t, \qquad \varepsilon_t \sim \text{i.i.d. } (0, \sigma^2),$$

where $\phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ and $\theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$. When $Z_t = \log(Y_t)$ and $d = 1$, a non-zero mean in X_t corresponds to a **drift** in $\log(Y_t)$, i.e., a constant average growth component.

3 Part III: Prediction

Denote T the length of the series. We assume the residuals are Gaussian.

3.1 6. Confidence region for (X_{T+1}, X_{T+2})

Let $\hat{\mu} = (\hat{X}_{T+1}, \hat{X}_{T+2})^\top$ be the 2-step-ahead forecast mean vector and $\hat{\Sigma}$ the 2x2 forecast error covariance matrix. Under Gaussian residuals, the joint forecast error is bivariate normal, so the level-α confidence region is the ellipse

$$\mathcal{C}_\alpha = \left\{ x \in \mathbb{R}^2 : (x - \hat{\mu})^\top \hat{\Sigma}^{-1} (x - \hat{\mu}) \le \chi^2_{2,\alpha} \right\}.$$

For an ARMA model, let $\{\psi_j\}_{j \ge 0}$ be the coefficients of the MA(∞) representation. Then

- $\mathrm{Var}(X_{T+1} - \hat{X}_{T+1}) = \sigma^2$,
- $\mathrm{Var}(X_{T+2} - \hat{X}_{T+2}) = \sigma^2(1 + \psi_1^2)$,
- $\mathrm{Cov}(X_{T+1} - \hat{X}_{T+1}, X_{T+2} - \hat{X}_{T+2}) = \sigma^2 \psi_1$,

where σ^2 is the innovation variance.

Interpretation. This region is the multivariate analogue of a univariate prediction interval: it is a joint set such that $\mathbb{P}((X_{T+1}, X_{T+2}) \in \mathcal{C}_\alpha) \approx \alpha$ under the model. The chi-square threshold comes from the quadratic form of a bivariate normal vector. In practice, if residuals are heavier-tailed than Gaussian, the ellipse may be too small (undercoverage).

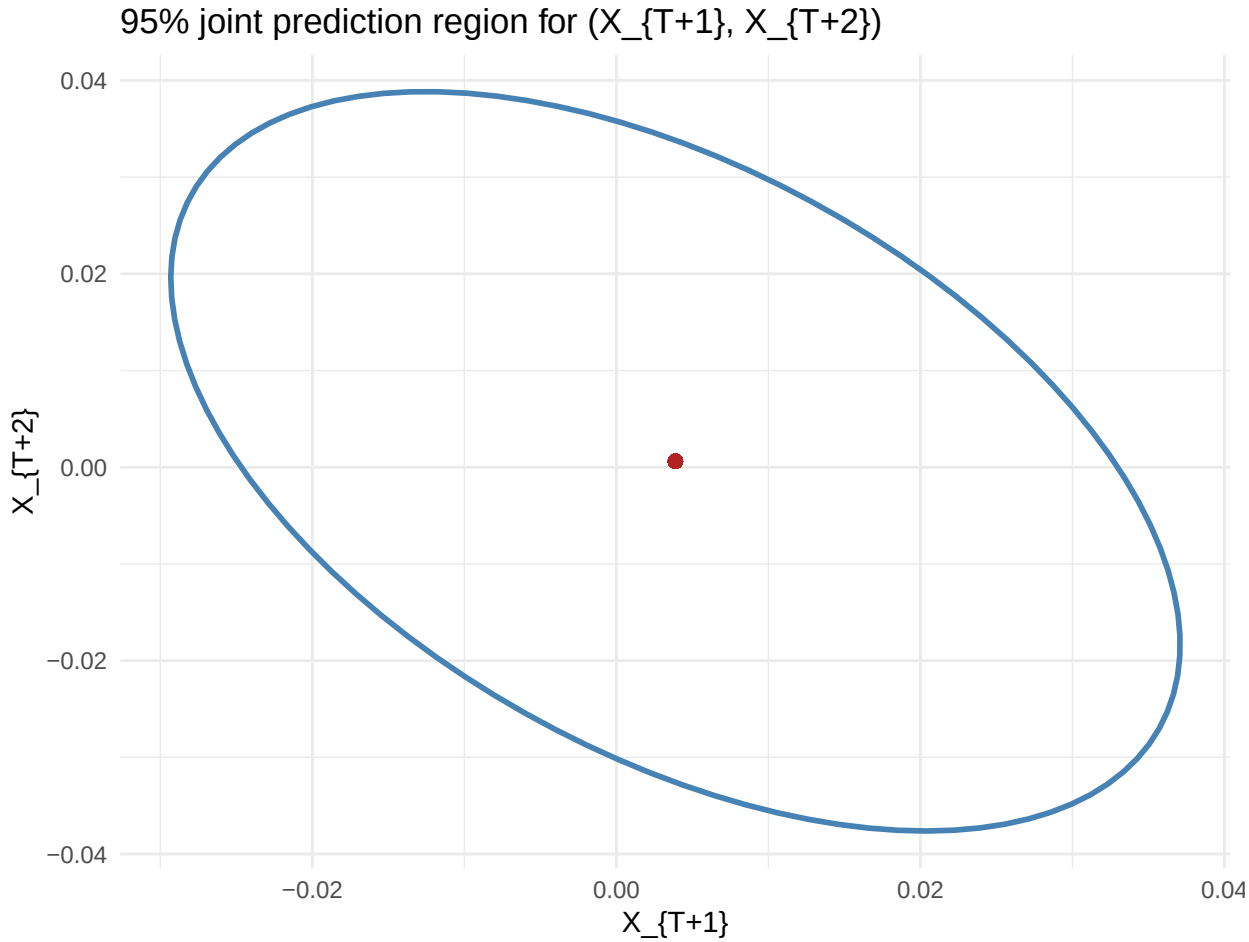
3.2 7. Hypotheses used to obtain this region

1. The ARMA model for X_t is correctly specified.
2. Innovations are i.i.d. Gaussian with variance σ^2.
3. Model parameters are treated as known (estimation uncertainty ignored for the region).
4. The process is stationary and invertible (so the MA(∞) representation exists).

Role of the assumptions.

- If the ARMA is misspecified, $\hat{\Sigma}$ is wrong and the region is not calibrated.
- Gaussianity is what makes the contour exactly elliptical with a chi-square cutoff; without it, ellipses are at best an approximation.
- Treating parameters as known ignores estimation uncertainty and typically makes the region slightly too optimistic; this effect is non-negligible in small samples but tends to shrink as T grows.

3.3 8. Graphical representation for $\alpha = 95\%$



Comments on the 95% region.

- The forecast-error correlation is -0.496, i.e., **negative**: a positive error at horizon 1 tends to be associated with a negative error at horizon 2 under the fitted ARMA dynamics.
- Using $\rho = \psi_1/\sqrt{1+\psi_1^2}$, this corresponds to $\psi_1 \approx \rho/\sqrt{1-\rho^2}$, hence here $\psi_1 \approx -0.571$.
- The ellipse is therefore tilted downward; its size is governed by the innovation variance $\sigma^2 = 1.84 \times 10^{-4}$ (on the X_t scale) and by $|\psi_1|$.

3.4 9. Open question

Let Y_t be a stationary time series observed from $t = 1$ to T, and suppose Y_{T+1} is observed before X_{T+1}. Using Y_{T+1} improves the prediction of X_{T+1} if and only if Y_{T+1} contains incremental predictive information about X_{T+1} beyond the sigma-field generated by $\{X_1, \ldots, X_T\}$. Formally, if

$$\mathbb{E}[X_{T+1} \mid X_1, \ldots, X_T, Y_{T+1}] \neq \mathbb{E}[X_{T+1} \mid X_1, \ldots, X_T],$$

then including Y_{T+1} improves the forecast in mean-square error.

A practical test is to estimate an ARMAX-type model for X_t with Y_t (or a proxy for Y_{T+1} if only available at $T+1$) and test the significance of the coefficient(s) on Y. This can be done via a t-test on the added regressor(s), or a likelihood-ratio test comparing the ARMA model for X_t against the ARMAX model.

Conditions and practical test.

- A convenient equivalent condition is that Y_{T+1} is not conditionally independent of X_{T+1} given past X: informally, Y must have **incremental predictive content** for X.
- In linear/Gaussian settings, this amounts to Y being correlated with the one-step-ahead innovation (forecast error) of X when conditioning on $\mathcal{F}_T = \sigma(X_1, \ldots, X_T)$.
- Practical implementation: fit an ARMAX model $X_t = \text{ARMA terms} + \beta Y_t + \varepsilon_t$ (or a lead/nowcast proxy for Y_{T+1}) and test $H_0 : \beta = 0$ (t-test / Wald) or compare log-likelihoods (LR test). A rejection indicates that observing Y_{T+1} can reduce the MSE of forecasting X_{T+1}.

4 Appendix: Code

```
knitr::opts_chunk$set(echo = FALSE, warning = FALSE, message = FALSE)



required_pkgs <- c(

  "readr", "dplyr", "ggplot2", "lubridate",

  "forecast", "tseries", "ellipse", "knitr"

)

missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]

if (length(missing_pkgs) > 0) {

  options(repos = c(CRAN = "https://cloud.r-project.org"))

  install.packages(missing_pkgs)

}



library(readr)

library(dplyr)

library(ggplot2)

library(lubridate)

library(forecast)

library(tseries)

library(ellipse)

library(knitr)
```

```r
meta_candidates <- list.files("data", pattern = "caract", full.names = TRUE)
if (length(meta_candidates) == 0) {
  stop("Metadata file not found in data/.")
}
meta_path <- meta_candidates[1]
meta <- read_delim(
  meta_path,
  delim = ";",
  col_names = TRUE,
  show_col_types = FALSE
)
names_norm <- iconv(names(meta), from = "", to = "ASCII//TRANSLIT")
pick_col <- function(pattern) {
  idx <- which(grepl(pattern, names_norm, ignore.case = TRUE))
  if (length(idx) == 0) return(NA_character_)
  names(meta)[idx[1]]
}

cols <- c(
  pick_col("idbank"),
  pick_col("mise"),
  pick_col("period"),
  pick_col("unit"),
  pick_col("activ"),
  pick_col("correct"),
  pick_col("indicat"),
  pick_col("zone"),
```

```
  pick_col("base")
)
cols <- cols[!is.na(cols)]


meta %>%
  slice(1) %>%
  select(all_of(cols)) %>%
  kable()
values_path <- file.path("data", "valeurs_mensuelles.csv")
raw <- read_delim(
  values_path,
  delim = ";",
  col_names = FALSE,
  show_col_types = FALSE
)


row_label <- raw[[1]]
row_label_norm <- iconv(row_label, from = "", to = "ASCII//TRANSLIT")
row_label_norm <- trimws(row_label_norm)
idx_updates <- which(row_label_norm == "Mises a jour")[1]
idx_period <- which(row_label_norm == "Periode")[1]
if (is.na(idx_updates) || is.na(idx_period)) {
  stop("Could not locate 'Mises a jour' or 'Periode' rows in the values file.")
}


updates_row <- raw[idx_updates, , drop = FALSE]
update_strings <- as.character(unlist(updates_row[-1]))
```

```r
update_dates <- suppressWarnings(lubridate::dmy_hm(update_strings, quiet = TRUE))


value_cols <- which(!is.na(update_dates)) + 1

vintage_tbl <- tibble(

  vintage_id = seq_along(value_cols),

  update_date = update_dates[!is.na(update_dates)],

  value_col = value_cols

) %>% arrange(update_date)


vintage_tbl %>% tail(6) %>% kable()


latest_vintage <- vintage_tbl %>% slice_max(update_date, n = 1)

chosen_col <- latest_vintage$value_col


series_raw <- raw[(idx_period + 1):nrow(raw), , drop = FALSE] %>%

  transmute(

    period = .data[[names(raw)[1]]],

    value = .data[[names(raw)[chosen_col]]]

  ) %>%

  mutate(

    date = as.Date(paste0(period, "-01")),

    value = readr::parse_number(value, locale = locale(decimal_mark = "."))

  ) %>%

  filter(!is.na(date))


start_year <- year(min(series_raw$date))

start_month <- month(min(series_raw$date))
```

```r
Y <- ts(series_raw$value, start = c(start_year, start_month), frequency = 12)
Y <- stats::na.omit(Y)

summary(series_raw$value)
use_log <- all(Y > 0, na.rm = TRUE)
Z <- if (use_log) log(Y) else Y

# Suggested differencing from ADF and KPSS
nd_adf <- forecast::ndiffs(Z, alpha = 0.05, test = "adf")
nd_kpss <- forecast::ndiffs(Z, alpha = 0.05, test = "kpss")
d <- max(nd_adf, nd_kpss)

X <- diff(Z, differences = d)

adf_Z <- tseries::adf.test(Z)
kpss_Z <- tseries::kpss.test(Z, null = "Level")

adf_X <- tseries::adf.test(X)
kpss_X <- tseries::kpss.test(X, null = "Level")

list(
  use_log = use_log,
  d_adf = nd_adf,
  d_kpss = nd_kpss,
  d_chosen = d,
  adf_Z_p = adf_Z$p.value,
```

```
  kpss_Z_p = kpss_Z$p.value,
  adf_X_p = adf_X$p.value,
  kpss_X_p = kpss_X$p.value
)
op <- par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
plot(Y, main = "Raw series Y_t", xlab = "Time", ylab = "Index")
plot(X, main = "Stationary series X_t", xlab = "Time", ylab = "Transformed index")
par(op)
op <- par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
forecast::Acf(X, lag.max = 36, main = "ACF of X_t")
forecast::Pacf(X, lag.max = 36, main = "PACF of X_t")
par(op)
max_p <- 5
max_q <- 5

grid <- expand.grid(p = 0:max_p, q = 0:max_q) %>%
  rowwise() %>%
  mutate(
    fit = list(tryCatch(
      Arima(X, order = c(p, 0, q), include.mean = TRUE),
      error = function(e) NULL
    )),
    aic = ifelse(is.null(fit), NA, AIC(fit)),
    bic = ifelse(is.null(fit), NA, BIC(fit))
  ) %>%
  ungroup() %>%
  arrange(aic)
```

```r
grid %>%
  filter(!is.na(aic)) %>%
  select(p, q, aic, bic) %>%
  head(5) %>%
  kable()

best_row <- grid %>% filter(!is.na(aic)) %>% slice(1)
best_fit <- best_row$fit[[1]]
best_row %>% select(p, q, aic, bic) %>% kable()
coef <- best_fit$coef
se <- sqrt(diag(best_fit$var.coef))
coef_tbl <- tibble::tibble(
  term = names(coef),
  estimate = unname(coef),
  std_error = unname(se),
  t_value = unname(coef / se)
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

coef_tbl %>% kable()
checkresiduals(best_fit, test = FALSE)
res <- residuals(best_fit)
lb_test <- Box.test(res, lag = 24, type = "Ljung-Box", fitdf = sum(best_fit$arma[1:4]))
jb_test <- tseries::jarque.bera.test(res)
alpha <- 0.95
```

```r
pred <- predict(best_fit, n.ahead = 2)

mu <- as.numeric(pred$pred)


# Extract AR and MA coefficients for psi1

ar <- if (!is.null(best_fit$model$phi)) best_fit$model$phi else numeric(0)

ma <- if (!is.null(best_fit$model$theta)) best_fit$model$theta else numeric(0)

psi1 <- if (length(ar) == 0 && length(ma) == 0) 0 else ARMAtoMA(ar, ma, lag.max = 1)[1]


sigma2 <- best_fit$sigma2

Sigma <- matrix(

  c(sigma2, sigma2 * psi1, sigma2 * psi1, sigma2 * (1 + psi1^2)),

  nrow = 2

)


ell <- as.data.frame(ellipse::ellipse(Sigma, centre = mu, level = alpha))

colnames(ell) <- c("X_T1", "X_T2")


corr12 <- Sigma[1, 2] / sqrt(Sigma[1, 1] * Sigma[2, 2])


p <- ggplot(ell, aes(x = X_T1, y = X_T2)) +

  geom_path(color = "steelblue", linewidth = 1) +

  geom_point(aes(x = mu[1], y = mu[2]), color = "firebrick", size = 2) +

  labs(

    title = "95% joint prediction region for (X_{T+1}, X_{T+2})",

    x = "X_{T+1}",

    y = "X_{T+2}"

  ) +
```

```
  theme_minimal()



p

code_chunks <- knitr::knit_code$get()

code_text <- unlist(code_chunks, use.names = FALSE)



cat("```r\n")

cat(paste(code_text, collapse = "\n\n"))

cat("\n```\n")
```